

Meta-Prompt-Maitre

Prompt-Engineering-Kit v4.0

Document de reference canonique pour la regeneration, le maintien, l'extension et le diagnostic de l'application PEK v4.0. Ce document capture l'integralite de la specification fonctionnelle, technique, architecturale et methodologique.

Super-Agent

Hybride CoT

Chaining

Full-Stack

Table des matieres

1. Identité & Philosophie	5
1.1 Définition	5
1.2 Principe Fondamental	5
1.3 Méthodologie : SUPER-AGENT HYBRIDE CoT + CHAINING	5
1.4 Langue	5
2. Architecture Système	5
2.1 Stack Technique	5
2.2 Structure des Fichiers	6
2.3 Schéma Routage UI	6
2.4 Topologie Réseau	6
3. Modèle de Données (Prisma)	7
3.1 Schéma Complet	7
3.2 Relations Clés	7
4. État Client (Zustand Store)	7
4.1 Types Exportés	7
4.2 État Global	7
4.3 Actions Principales	8
4.4 Constantes Exportées	8
5. Pipeline Chain-of-Thought — Spécification Complète	8
5.1 Vue d'Ensemble	8
5.2 Détail de Chaque Étape	8
Étape 1 — COMPRÉHENSION	8
Étape 2 — DÉCOMPOSITION	9
Étape 3 — EXTRACTION	9
Étape 4 — STRUCTURATION	9
Étape 5 — GÉNÉRATION	9
Étape 6 — VALIDATION	9

Étape 7 — AMÉLIORATION	10
5.3 System Prompt du Pipeline	10
5.4 Construction du Message Utilisateur	10
5.5 Appel LLM	10
5.6 Gestion des Erreurs dans le Pipeline	10
6. Blocs Modulaires A-J	11
6.1 Tableau de Référence	11
6.2 Règles de Sélection	11
7. API REST — Endpoints	12
7.1 Sessions	12
`GET /api/sessions`	12
`POST /api/sessions`	12
`GET /api/sessions/[id]`	12
`DELETE /api/sessions/[id]`	12
`POST /api/sessions/[id]/execute`	12
`GET /api/sessions/[id]/steps/[stepNumber]`	12
`PATCH /api/sessions/[id]/steps/[stepNumber]`	13
7.2 Templates	13
`GET /api/templates`	13
`POST /api/templates`	13
`GET /api/templates/[id]` / `DELETE /api/templates/[id]`	13
7.3 Validation	13
`POST /api/validate`	13
7.4 Export	13
`POST /api/export`	13
7.5 Format de Réponse Standardisé	13
8. Composants UI — Spécification Détaillée	13
8.1 `PekHeader`	13
8.2 `PekSessionForm`	14
8.3 `PekCoTPipeline`	14

8.4 `PekStepOutput`	14
8.5 `PekSessionsList`	14
8.6 `PekTemplatesGallery`	15
8.7 `PekGuide`	15
8.8 `PekValidationReport`	15
8.9 `PekFooter`	15
9. Système de Validation & Scoring	16
9.1 Barème	16
9.2 Évaluation Automatique (Étape 6 du Pipeline)	16
9.3 Évaluation Algorithmique (`POST /api/validate`)	16
9.4 Checks de Validation	16
9.5 Grades	17
10. Templates & Modèles Réutilisables	17
10.1 Templates par Défaut	17
10.2 Modèle de Données Template (DB)	17
10.3 Création de Template Custom	17
11. Design System & Contraintes Visuelles	18
11.1 Palette de Couleurs	18
11.2 Typographie	18
11.3 Animations (Framer Motion)	18
11.4 Responsive	18
11.5 Règles de Layout	18
12. Flux de Données End-to-End	19
12.1 Création de Session + Pipeline	19
12.2 Consultation d'une Session Passée	19
13. Règles d'Implémentation Inviolables	19
13.1 Règles Architecturales	19
13.2 Règles de Code	19
13.3 Règles UX/UI	19
13.4 Règles Métier PEK	19

14. Glossaire Technique	20
15. Matrice de Diagnostic	20
15.1 Problèmes Frontend	20
15.2 Problèmes Backend	21
15.3 Problèmes de Données	21
Annexe A : Formule de Mapping Store ↔ DB	21
Annexe B : Structure du Markdown Export	21

1. Identité & Philosophie

1.1 Définition

Prompt-Engineering-Kit v4.0 (PEK v4.0) est une application full-stack de génération automatisée de kits de prompt engineering. Elle transforme tout projet IA en un kit structuré, réutilisable et validé, via une méthodologie **SUPER-AGENT HYBRIDE CoT + CHAINING**.

1.2 Principe Fondamental

L'application ne génère pas un simple prompt — elle **décompose un projet source en blocs modulaires normalisés (A-J)**, les traite séquentiellement via un **pipeline Chain-of-Thought à 7 étapes**, et produit un **kit complet, scoré et exportable**.

1.3 Méthodologie : SUPER-AGENT HYBRIDE CoT + CHAINING

- **CoT (Chain-of-Thought)** : Chaque étape du pipeline raisonne logiquement sur la précédente. Les sorties s'accumulent et se nourrissent mutuellement.
- **Chaining** : Les 7 étapes s'exécutent de manière **strictement séquentielle**. L'étape N+1 reçoit toujours les sorties des étapes 1 à N.
- **Hybride** : Combine le raisonnement LLM (génération de contenu) avec des mécanismes déterministes (validation par règles, scoring algorithmique).
- **Super-Agent** : Le système agit comme un architecte expert autonome qui orchestre l'ensemble du processus.

1.4 Langue

L'application est **en français** par défaut (`lang="fr"` sur `<html>`). Les contenus générés sont en français sauf configuration contraire (champ `language: "en"`).

2. Architecture Système

2.1 Stack Technique

Couche	Technologie	Version / Notes
Framework	Next.js 16 (App Router)	TypeScript 5 strict
Styling	Tailwind CSS 4	Variables CSS intégrées
UI Components	shadcn/ui (New York style)	Lucide icons
State Management	Zustand	Store centralisé (<code>src/lib/store.ts</code>)
ORM	Prisma	SQLite (client only)
LLM Backend	z-ai-web-dev-sdk	<code>ZAI.create()</code> → <code>zai.chat.completions.create()</code>
Animations	Framer Motion	Transitions de page et micro-interactions
Forms	react-hook-form + zod/v4	Validation côté client
Theme	next-themes	Light/Dark, default dark
Markdown	react-markdown	Rendu des sorties d'étapes
Notifications	Sonner (Toaster)	Toasts de feedback
Dates	date-fns	Formatage FR

2.2 Structure des Fichiers

```
src/ └─ app/ | └─ layout.tsx # Root layout : ThemeProvider, Toaster, fonts Geist | └─ page.tsx # SPA
unique : routing par onglets (tabs) | └─ globals.css # Styles globaux Tailwind + custom scrollbar | └─
api/ | └─ sessions/ | | └─ route.ts # GET (liste), POST (création + 7 steps) | | └─ [id]/ | | └─
route.ts # GET (détail), DELETE | | └─ execute/route.ts # POST : Pipeline CoT complet (7 appels LLM) | |
└─ steps/[stepNumber]/ | | └─ route.ts # GET (polling), PATCH (mise à jour) | └─ templates/ | | └─
route.ts # GET (liste), POST (création) | | └─ [id]/route.ts # GET, DELETE | └─ validate/route.ts # POST
: validation algorithmique | └─ export/route.ts # POST : export Markdown | └─ components/ | └─
pek-header.tsx # Navigation sticky + dark mode toggle + mobile sheet | └─ pek-footer.tsx # Footer sticky
avec branding | └─ pek-session-form.tsx # Formulaire de création + lancement pipeline + polling | └─
pek-cot-pipeline.tsx # Stepper visuel 7 étapes (horizontal desktop, vertical mobile) | └─
pek-step-output.tsx # Rendu Markdown d'une étape + copie | └─ pek-sessions-list.tsx # Historique des
sessions + voir/supprimer | └─ pek-templates-gallery.tsx # Galerie de templates + création custom | └─
pek-guide.tsx # Guide interactif (accordeons méthodologie + blocs) | └─ pek-validation-report.tsx #
Rapport de validation avec score circulaire | └─ theme-provider.tsx # Wrapper next-themes | └─ ui/ #
Composants shadcn/ui standards | └─ lib/ | └─ store.ts # Zustand store (types, constantes, actions) | └─ db.ts
# Prisma client singleton | └─ utils.ts # Utilitaire cn() pour className merging
```

2.3 Schéma Routage UI

L'application est une **Single Page Application** avec un seul route visible (/). La navigation se fait via un système d'onglets géré par le Zustand store :

Clé `activeTab`	Composant	Description
"session"	PekSessionForm + pipeline	Nouvelle session + formulaire + pipeline + output + validation
"sessions"	PekSessionsList	Historique des sessions passées
"templates"	PekTemplatesGallery	Galerie de modèles réutilisables
"guide"	PekGuide	Guide méthodologique interactif

2.4 Topologie Réseau

- Port unique exposé : **3000** (dev server Next.js)
- Gateway Caddy pour le routage inter-services
- Les requêtes vers des mini-services passent par **?XTransformPort=XXXX** (query parameter)
- LLM calls via **z-ai-web-dev-sdk** côté serveur uniquement (jamais côté client)

3. Modèle de Données (Prisma)

3.1 Schéma Complet

```
datasource db { provider = "sqlite" url = env("DATABASE_URL") } model Project { id String @id @default(cuid()) name String description String? projectType String @default("web") status String @default("draft") createdAt DateTime @default(now()) updatedAt DateTime @updatedAt sessions Session[] } model Session { id String @id @default(cuid()) title String projectId String project Project @relation(fields: [projectId], references: [id], onDelete: Cascade) currentStep Int @default(1) status String @default("in_progress") // in_progress | completed | error context String @default("") // Description du contexte sourceMaterial String @default("") // Contenu source du projet results String @default("") // Résultats attendus constraints String @default("") // Contraintes additionnelles iterations String @default("") environment String @default("") inputUrl String? inputFormat String @default("markdown") // markdown | json | yaml | code | url | pdf selectedBlocks String @default("A,C,E,G,H") // Liste séparée par virgules complexity String @default("moyen") // simple | moyen | complexe language String @default("fr") // fr | en createdAt DateTime @default(now()) updatedAt DateTime @updatedAt steps Step[] validations Validation[] } model Step { id String @id @default(cuid()) sessionId String session Session @relation(fields: [sessionId], references: [id], onDelete: Cascade) stepNumber Int // 1-7 name String // Nom étape (COMPRÉHENSION, DÉCOMPOSITION, etc.) status String @default("pending") // pending | running | completed | error | skipped output String @default("") // Sortie Markdown de l'étape duration Int @default(0) // Durée en secondes createdAt DateTime @default(now()) updatedAt DateTime @updatedAt } model Validation { id String @id @default(cuid()) sessionId String session Session @relation(fields: [sessionId], references: [id], onDelete: Cascade) completeness Int @default(0) // 0-10 coherence Int @default(0) // 0-8 quality Int @default(0) // 0-7 totalScore Int @default(0) // 0-25 maxScore Int @default(25) checks String @default("[]") // JSON : Array<{label: string; passed: boolean}> passed Boolean @default(false) createdAt DateTime @default(now()) updatedAt DateTime @updatedAt } model Template { id String @id @default(cuid()) name String description String systemPrompt String blocks String @default("A,C,E,G,H") // Liste séparée par virgules isDefault Boolean @default(false) createdAt DateTime @default(now()) updatedAt DateTime @updatedAt }
```

3.2 Relations Clés

- **Project 1→N Session** : Un projet contient plusieurs sessions. Suppression en cascade.
- **Session 1→N Step** : Chaque session a exactement 7 étapes (créées à l'initialisation).
- **Session 1→N Validation** : Chaque session peut avoir plusieurs validations (historique).
- **Step appartient à Session** : Cascade delete.

4. État Client (Zustand Store)

4.1 Types Exportés

```
type BlockId = "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J"; interface Project { id: string; name: string; description: string; createdAt: string; updatedAt: string; } interface Session { id: string; projectId: string; title: string; status: "pending" | "running" | "completed" | "error"; complexity: "simple" | "moyen" | "complexe"; score: number | null; createdAt: string; completedAt: string | null; } interface Step { id: number; name: string; description: string; status: "pending" | "active" | "done" | "error"; duration: number | null; } interface BlockConfig { id: BlockId; name: string; description: string; required: boolean; recommended: boolean; } interface Template { id: string; name: string; description: string; blocks: BlockId[]; systemPrompt: string; icon: string; }
```

4.2 État Global

```
{ // Navigation activeTab: string; // "session" | "sessions" | "templates" | "guide" sidebarOpen: boolean; // Données projects: Project[]; sessions: Session[]; currentSessionId: string | null; // Pipeline steps: Step[]; // 7 étapes (DEFAULT_STEPS) currentStep: number; // Index 0-6 stepOutputs: Record<number, string>; // index → contenu Markdown isGenerating: boolean; // Configuration selectedBlocks: BlockId[]; showTemplateDialog: boolean; // Validation validationScore: number | null; // 0-25 validationDetails: { completeness: number; // 0-10 coherence: number; // 0-8 quality: number; // 0-7 checks: { label: string; passed: boolean }[]; } | null; }
```


4.3 Actions Principales

Action	Description
<code>setActiveTab(tab)</code>	Change l'onglet actif
<code>resetPipeline()</code>	Réinitialise steps, stepOutputs, currentStep, isGenerating, validation, currentSessionId
<code>setSteps(steps)</code>	Met à jour le tableau des 7 étapes
<code>setStepOutput(index, output)</code>	Enregistre la sortie Markdown d'une étape
<code>setCurrentStep(index)</code>	Change l'étape actuellement visualisée
<code>setIsGenerating(bool)</code>	Active/désactive l'état de génération
<code>toggleBlock(block)</code>	Ajoute/retire un bloc (les blocs required ne peuvent pas être retirés)
<code>setValidationScore(score)</code>	Enregistre le score total
<code>setValidationDetails(details)</code>	Enregistre le détail de validation
<code>setSessions(sessions)</code>	Met à jour la liste des sessions
<code>setCurrentSessionId(id)</code>	Enregistre l'ID de la session courante

4.4 Constantes Exportées

DEFAULT_STEPS** : Les 7 étapes par défaut du pipeline CoT.

BLOCK_CONFIGS** : Configuration des 10 blocs A-J avec nom, description, required/recommended.

DEFAULT_TEMPLATES** : 5 templates prédéfinis (Web App, Agent IA, Pipeline Data, Recherche, Formation).

5. Pipeline Chain-of-Thought — Spécification Complète

5.1 Vue d'Ensemble

Le pipeline est le cœur de l'application. Il transforme une session (projet + configuration) en un kit structuré via 7 appels LLM séquentiels avec accumulation de contexte.

[Session Input] → Étape 1 → Étape 2 → ... → Étape 7 → [Kit Final + Score] ↑ ↑ — Chaining : chaque étape reçoit les sorties des précédentes

5.2 Détail de Chaque Étape

Étape 1 — COMPRÉHENSION

Objectif : Analyse en profondeur du projet fourni.

Sorties attendues :

- Objectif principal du projet IA
- Domaine d'application
- Technologies et frameworks utilisés
- Acteurs impliqués
- Niveau de complexité global
- Points clés à préserver

Format : Analyse structurée avec sections claires.

Étape 2 — DÉCOMPOSITION

Objectif : Découpage du projet en composants réutilisables.

Sorties attendues :

- Responsabilité principale de chaque composant
- Entrées/sorties définies
- Dépendances inter-composants
- Évaluation de réutilisabilité
- Priorité de conversion en bloc prompt

Format : Liste structurée des composants avec caractéristiques.

Étape 3 — EXTRACTION

Objectif : Extraction des patterns récurrents et éléments réutilisables.

Sorties attendues :

- Patterns de conception identifiés
- Structures de données communes
- Workflows récurrents
- Contraintes techniques récurrentes
- Bonnes pratiques observées
- Exemples concrets dans **4+ domaines différents**

Format : Catalogue des patterns avec exemples concrets.

Étape 4 — STRUCTURATION

Objectif : Organisation des éléments extraits en structure PEK cohérente.

Sorties attendues :

- Mapping de chaque élément aux blocs A-J
- Relations entre blocs
- Ordre recommandé d'utilisation
- Blocs obligatoires vs optionnels
- Plan d'assemblage du kit

Format : Structure hiérarchique avec mappage aux blocs.

Étape 5 — GÉNÉRATION

Objectif : Rédaction du contenu complet pour chaque bloc sélectionné.

Sorties attendues :

- Contenu détaillé et opérationnel par bloc
- Exemples concrets variés (4+ domaines)
- Placeholders pour les variables
- Cas d'erreur et traitements
- Rédaction dans la langue configurée

Format : Contenu complet de chaque bloc en Markdown structuré.

Étape 6 — VALIDATION

Objectif : Évaluation qualité et complétude du kit généré.

Critères évalués :

- **Complétude** (10 pts) : Tous les blocs sélectionnés présents et complets ?
- **Cohérence** (8 pts) : Références entre blocs correctes ?
- **Qualité** (7 pts) : Exemples pertinents et variés (4+ domaines) ?

Sorties attendues :

- Note par critère avec commentaire justifié
- Points d'amélioration prioritaires

Format : Rapport de validation structuré avec scores.

Étape 7 — AMÉLIORATION

Objectif : Corrections et optimisations finales basées sur la validation.

Actions :

- Correction des incohérences détectées
- Enrichissement des exemples (objectif 4+ domaines)
- Optimisation des formulations
- Ajout des éléments manquants
- Production du kit final versionné

Format : Kit PEK final complet en Markdown, prêt à l'emploi, avec numéro de version.

5.3 System Prompt du Pipeline

Tu es un Architecte en Prompt Engineering spécialisé en conversion de projets IA en kits structurés réutilisables. Méthodologie: Chain-of-Thought en 7 étapes - Étape 1: COMPRÉHENSION – Analyse approfondie du projet - Étape 2: DÉCOMPOSITION – Découpage en composants - Étape 3: EXTRACTION – Identification des patterns réutilisables - Étape 4: STRUCTURATION – Organisation en blocs PEK - Étape 5: GÉNÉRATION – Rédaction du contenu des blocs - Étape 6: VALIDATION – Évaluation qualité et complétude - Étape 7: AMÉLIORATION – Corrections et optimisations finales Contraintes: 1. Préservation de l'original – Ne jamais altérer le sens du projet source 2. Complétude des fichiers – Chaque bloc sélectionné doit être exhaustif 3. Cohérence des références – Les cross-références entre blocs doivent être exactes 4. Respect des schemas – Suivre les formats définis pour chaque bloc 5. Application du Chain-of-Thought – Raisonnement étape par étape, logique et traçable 6. Qualité des exemples (4+ domaines) – Varier les domaines d'application 7. Automatisation et versionnage – Prévoir l'intégration en pipeline CI/CD 8. Pas de greenwashing – Pas de fausses promesses, rester factuel et mesurable 9. Intégration du feedback – Prévoir des mécanismes de retour et d'itération

5.4 Construction du Message Utilisateur

Pour chaque étape N, le message utilisateur est construit ainsi :

```
## Contexte de la session {session.context} ## Matériel source {session.sourceMaterial} ## Résultats attendus {session.results} ## Contraintes additionnelles {session.constraints} ## Résultats des étapes précédentes ### Étape 1 – COMPRÉHENSION {output_étape_1} ### Étape 2 – DÉCOMPOSITION {output_étape_2} ... --- {instruction_de_l_étape_courante}
```

5.5 Appel LLM

```
const zai = await ZAI.create(); const completion = await zai.chat.completions.create({ messages: [ { role: 'assistant', content: systemPrompt }, { role: 'user', content: userMessage }, ], thinking: { type: 'disabled' }, });
```

Note critique : Le system prompt est passé en tant que **role: 'assistant'** (convention du SDK), et le message utilisateur réel en **role: 'user'**.

5.6 Gestion des Erreurs dans le Pipeline

- Si une étape échoue : les étapes restantes sont marquées **skipped**, la session passe en **status: 'error'**.
- La réponse contient les steps réussis + l'erreur.

-
- Le client (polling) détecte le statut **error** et arrête le polling.
-

6. Blocs Modulaires A-J

6.1 Tableau de Référence

ID	Nom	Obligatoire	Recommandé	Description
A	Résumé Exécutif	✓		Vue d'ensemble du kit : objectifs, périmètre, points clés
B	Architecture & Composants			Architecture technique, composants, interactions, dépendances
C	Prompt / Instructions			Prompts principaux et instructions détaillées avec exemples
D	Données & Schemas			Structures de données, schémas de validation, formats
E	Contraintes & Règles			Règles métier, contraintes techniques, limites, garde-fous
F	Résultats & Outputs			Formats de sortie attendus, exemples concrets, cas d'utilisation
G	Guide d'Utilisation		✓	Instructions pas-à-pas, workflows typiques, bonnes pratiques
H	Recommandations	✓		Bonnes pratiques, optimisations, conseils d'expert, pièges
I	Tests & Validation			Cas de test, critères de validation, scénarios d'erreur
J	Glossaire & Références			Termes techniques, acronymes, sources externes

6.2 Règles de Sélection

- Minimum **2 blocs** requis pour lancer une session.
 - Les blocs **A (Résumé Exécutif)** et **H (Recommandations)** sont **obligatoires** — ils ne peuvent pas être désélectionnés dans le formulaire.
 - Le bloc **G (Guide d'Utilisation)** est **recommandé** mais facultatif.
 - Les blocs sélectionnés sont stockés comme chaîne séparée par virgules (ex: "**A,C,E,G,H**").
-

7. API REST — Endpoints

7.1 Sessions

GET /api/sessions

Description : Liste toutes les sessions avec leurs steps et validations.

Réponse :

```
{ "success": true, "data": [ { "id": "cuid...", "title": "Kit Agent de Support Client", "status": "completed", "steps": [...], "validations": [{ "totalScore": 22, ... }], "createdAt": "2025-01-15T10:30:00.000Z" } ] }
```

POST /api/sessions

Description : Crée une nouvelle session avec 7 étapes par défaut (status: pending).

Corps :

```
{ "title": "string (requis, min 3 car.)", "context": "string (description)", "sourceMaterial": "string (contenu source)", "results": "string (résultats attendus)", "constraints": "string (contraintes)", "inputFormat": "markdown | json | yaml | code | url | pdf", "selectedBlocks": "A,C,E,G,H", "complexity": "simple | moyen | complexe", "language": "fr | en" }
```

Comportement : Si aucun **projectId** n'est fourni, un projet par défaut est créé automatiquement.

Réponse : { success: true, data: { id, title, steps: [...7 steps], ... }, status: 201 }

GET /api/sessions/[id]

Description : Récupère une session avec ses steps et validations.

DELETE /api/sessions/[id]

Description : Supprime une session en cascade (steps + validations).

POST /api/sessions/[id]/execute

Description : **ENDPOINT CRITIQUE** — Exécute le pipeline CoT complet (7 appels LLM séquentiels).

Comportement :

1. Charge la session avec ses steps
2. Construit le system prompt PEK
3. Pour chaque step (1→7) :
 - a. Marque le step comme **running** en DB
 - b. Construit le user message avec contexte + sorties précédentes
 - c. Appelle le LLM via **z-ai-web-dev-sdk**
 - d. Sauvegarde le output et la durée en DB
 - e. Accumule le output pour les étapes suivantes
4. Après l'étape 6 : calcule et sauvegarde la validation
5. Marque la session comme **completed**

Erreurs : Si une étape échoue, les suivantes sont marquées **skipped**, la session passe en **error**.

GET /api/sessions/[id]/steps/[stepNumber]

Description : Récupère le statut et la sortie d'une étape (utilisé pour le **polling**).

Paramètres : **stepNumber** de 1 à 7.

Réponse :

```
{ "success": true, "data": { "id": "cuid...", "stepNumber": 3, "name": "EXTRACTION", "status": "running", // pending | running | completed | error | skipped "output": "...", "duration": 12 } }
```

PATCH /api/sessions/[id]/steps/[stepNumber]

Description : Mise à jour manuelle du statut/sortie d'une étape.

7.2 Templates

GET /api/templates

Description : Liste tous les templates (defaults en premier).

POST /api/templates

Description : Crée un template personnalisé.

Corps :

```
{ "name": "string (requis)", "description": "string", "systemPrompt": "string (requis)", "blocks": "A,C,E,G,H" }
```

GET /api/templates/[id] / DELETE /api/templates/[id]

Description : Récupère / Supprime un template.

7.3 Validation

POST /api/validate

Description : Validation algorithmique d'un contenu PEK (sans appel LLM).

Corps :

```
{ "content": "string (contenu Markdown à valider)", "blocks": ["A", "C", "E", "G", "H"] }
```

Réponse :

```
{ "success": true, "data": { "completeness": 8, // 0-10 "coherence": 7, // 0-8 "quality": 6, // 0-7 "totalScore": 21, // 0-25 "maxScore": 25, "grade": "B", // A | B | C | D "checks": [ { "label": "Complétude des blocs (4/5)", "passed": true }, ... ], "passedCount": 10, "totalChecks": 12, "passed": true } }
```

7.4 Export

POST /api/export

Description : Exporte une session complète en Markdown.

Corps : { "sessionId": "string" }

Réponse :

```
{ "success": true, "data": { "sessionId": "...", "title": "Kit Agent de Support Client", "markdown": "# Prompt-Engineering-Kit v4.0 – Kit Agent...", "filename": "Kit_Agent_de_Support_Client_PEK.md" } }
```

7.5 Format de Réponse Standardisé

Tous les endpoints retournent :

- Succès : { success: true, data: <payload> }
- Erreur : { success: false, error: "message lisible" } avec code HTTP approprié (400, 404, 500)

8. Composants UI — Spécification Détaillée

8.1 PekHeader

- **Position** : Sticky en haut (sticky top-0 z-50), avec backdrop blur
- **Contenu** :
 - Logo (icône BrainCircuit dans un carré ambre) + titre "Prompt-Engineering-Kit" + badge "v4.0"
 - Sous-titre "Super-Agent Hybride CoT + Chaining"

- Navigation desktop (4 boutons : Nouvelle Session, Sessions, Modèles, Guide)
- Toggle dark/light mode (icônes `Sun/Moon`)
- Bouton "Nouvelle Session" (ambre, CTA principal)
- Menu mobile via `Sheet` (slide-in depuis la droite)
- **État actif** : Onglet actif highlighted avec `bg-amber-600/10 text-amber-500`

8.2 PekSessionForm

- **Role** : Composant le plus critique de l'application.
- **Structure** : 4 cartes (shadcn Card) dans un formulaire react-hook-form :
 1. **Informations de base** : Titre (Input), Description (Textarea), Contenu source (Textarea, min 120px)
 2. **Résultats & Contraintes** : Résultats attendus (Textarea), Contraintes (Textarea)
 3. **Configuration** : Format d'entrée (Select), Complexité (Select), Langue (Select) — grid 3 colonnes
 4. **Sélection des Blocs** : Checkboxes pour les 10 blocs A-J, badges "Obligatoire"/"Recommandé"
- **Validation Zod** : titre min 3, description min 10, sourceContent min 20
- **Soumission** :
 1. Vérifie ≥ 2 blocs sélectionnés
 2. `resetPipeline()` pour nettoyer l'état
 3. `POST /api/sessions` → crée la session
 4. Lance `pollStepStatus()` en arrière-plan (polling toutes les 2s)
 5. `POST /api/sessions/{id}/execute` → lance le pipeline
 6. À la fin : parse le score de validation depuis l'étape 6, met à jour le store

8.3 PekCoTPipeline

- **Affichage conditionnel** : Visible uniquement quand `steps.some(s => s.status !== "pending")`
- **Desktop** : Stepper horizontal avec ligne de connexion animée (Framer Motion), 7 cercles icônes
- **Mobile** : Stepper vertical avec cartes empilées (AnimatePresence)
- **Icônes par statut** :
 - `pending` → `Circle` (gris)
 - `active` → `Loader2` (ambre, rotation infinie)
 - `done` → `CheckCircle2` (vert)
 - `error` → `AlertCircle` (rouge)
- **Interactivité** : Cliquer sur une étape terminée affiche sa sortie
- **Timer** : Compteur de secondes pendant la génération

8.4 PekStepOutput

- **Affichage conditionnel** : Seulement si une étape a du contenu
- **Loading state** : Skeleton placeholders pendant la génération
- **Rendu** : `ReactMarkdown` avec styles prose personnalisés
- Code inline : fond muted, texte ambre monospace
- Code blocks : fond zinc-900, monospace, scroll horizontal
- Blockquotes : bordure gauche ambre
- **Actions** : Bouton "Copier" avec feedback visuel (coche verte 2s)

8.5 PekSessionsList

- **Chargement** : `useEffect` → `GET /api/sessions` au mount
- **Mapping** : Status DB → Status UI (`in_progress`→`running`, `completed`→`completed`)

-
- **Cartes de session** :
 - Titre + date formatée (date-fns, locale fr)
 - Badge de statut (coloré : ambre/vert/rouge)
 - Badge de complexité
 - Score (étoile ambre + valeur colorée selon seuil)
 - Boutons : "Voir" (charge la session dans le pipeline) / "Supprimer" (AlertDialog de confirmation)
 - **État vide** : Icône Inbox + texte incitatif + bouton "Rafraîchir"
 - **Scroll** : `max-h-[600px] overflow-y-auto` avec scrollbar custom

8.6 PekTemplatesGallery

- **Templates par défaut** : 5 templates codés en dur dans le store (`DEFAULT_TEMPLATES`)
- Kit Web App (Globe) — blocs A,B,C,D,E,F,G,H
- Kit Agent IA (Bot) — blocs A,C,D,E,F,G,H,I
- Kit Pipeline Data (Database) — blocs A,B,D,E,F,H,I,J
- Kit Recherche (BookOpen) — blocs A,C,D,F,H,I,J
- Kit Formation (GraduationCap) — blocs A,C,F,G,H,I,J
- **Création custom** : Dialog avec nom, description, system prompt, sélection de blocs
- **Action "Utiliser"** : Applique les blocs du template et navigue vers l'onglet session

8.7 PekGuide

- **Structure** : 4 accordeons (Accordion, type="multiple", 2 ouverts par défaut) :
- 1. **Méthodologie CoT** : 7 cartes décrivant chaque étape avec 4 sous-points
- 2. **Blocs A-J** : 10 cartes avec nom, badges, description
- 3. **Contraintes & Limites** : Contraintes techniques + limites connues
- 4. **Contrôle Qualité** : Barème 25pts (10+8+7), barème de qualité (A/B/C/D)

8.8 PekValidationReport

- **Affichage conditionnel** : Seulement si `validationScore` est non-null
- **Score circulaire** : SVG animé (Framer Motion), cercle progressif avec score au centre
- **Détails** : 3 barres de progression (Complétude /10, Cohérence /8, Qualité /7)
- **Checklist** : Grille de 7-12 contrôles avec icônes ✓/✗
- **Couleurs dynamiques** : ≥ 22 vert, ≥ 20 ambre, < 20 rouge
- **Action** : Bouton "Régénérer" si score < 22

8.9 PekFooter

- **Position** : Sticky en bas (`mt-auto`), bordure supérieure
 - **Contenu** : Icône BrainCircuit + texte branding + copyright avec année dynamique
-

9. Système de Validation & Scoring

9.1 Barème

Critère	Points Maximum	Description
Complétude	10	Tous les blocs sélectionnés présents et remplis
Cohérence	8	Cohérence interne et logique entre blocs
Qualité	7	Clarté, précision, utilité du contenu
TOTAL	25	

9.2 Évaluation Automatique (Étape 6 du Pipeline)

Après l'étape 6 (VALIDATION), le système calcule :

1. **Complétude** : Compte les blocs mentionnés dans la sortie de validation (**Bloc X, bloc X, X:, X :**) / blocs sélectionnés × 10.
2. **Cohérence** : Par défaut 8 (évaluation LLM).
3. **Qualité** : Par défaut 7 (évaluation LLM).
4. **Total** : Somme des trois.

9.3 Évaluation Algorithmique (POST /api/validate)

Validation déterministe sans LLM :

- **Complétude** : Détection de patterns textuels pour chaque bloc dans le contenu.
- **Cohérence** : Vérifie la présence de headings (**###, ##**), séparateurs (**---**), couverture de blocs ≥ 80%.
- **Qualité** : Vérifie la présence d'exemples, la longueur (>500, >2000), la présence de code blocks.

9.4 Checks de Validation

Liste des contrôles automatiques (12 items) :

1. Complétude des blocs (X/N)
2. Références croisées cohérentes
3. Exemples dans 4+ domaines
4. Préservation du contenu original
5. Fidélité au projet source
6. Structuration conforme PEK
7. Variables et placeholders définis
8. Cas d'erreur couverts
9. Instructions claires et non ambiguës
10. Formats de sortie spécifiés
11. Pas de greenwashing détecté
12. Mécanismes de feedback prévus

9.5 Grades

Score	Grade	Label	Couleur
≥ 22	A	Excellent, prêt à l'export	emerald-500
20 - 21	B	Acceptable, améliorations recommandées	amber-500
16 - 19	C	Moyen	—
< 16	D	Insuffisant, régénération requise	red-500

10. Templates & Modèles Réutilisables

10.1 Templates par Défaut

ID	Nom	Icône	Blocs	System Prompt
tpl-webapp	Kit Web App	Globe	A,B,C,D,E,F,G,H	"Tu es un expert en développement web..."
tpl-agent	Kit Agent IA	Bot	A,C,D,E,F,G,H,I	"Tu es un expert en agents IA..."
tpl-pipeline	Kit Pipeline Data	Database	A,B,D,E,F,H,I,J	"Tu es un expert en data engineering..."
tpl-recherche	Kit Recherche	BookOpen	A,C,D,F,H,I,J	"Tu es un chercheur expérimenté..."
tpl-formation	Kit Formation	GraduationCap	A,C,F,G,H,I,J	"Tu es un expert en pédagogie..."

10.2 Modèle de Données Template (DB)

```
model Template { id String @id @default(cuid()) name String // Nom affiché description String // Description courte systemPrompt String // Instructions système pour le LLM blocks String @default("A,C,E,G,H") // Blocs séparés par virgules isDefault Boolean @default(false) // Template système ? createdAt DateTime @default(now()) updatedAt DateTime @updatedAt }
```

10.3 Création de Template Custom

Via le Dialog dans `PekTemplatesGallery` :

- Nom (requis)
- Description
- System Prompt (textarea monospace)
- Sélection de blocs (checkboxes A-J)

Le template est stocké côté client dans le state local (non persisté en DB via la UI actuelle — l'API existe mais n'est pas encore connectée au composant).

11. Design System & Contraintes Visuelles

11.1 Palette de Couleurs

Élément	Couleur	Tailwind Class
Primaire	Ambre/Gold	<code>amber-500, amber-600, amber-700</code>
Succès	Émeraude	<code>emerald-500</code>
Erreur	Rouge	<code>red-500</code>
Fond	Background system	<code>bg-background</code>
Texte	Foreground system	<code>text-foreground</code>
Muted	System muted	<code>text-muted-foreground, bg-muted</code>
Interdits	Indigo, Bleu	✗ Jamais utilisés sauf demande explicite

11.2 Typographie

- **Font Sans** : Geist Sans (`--font-geist-sans`)
- **Font Mono** : Geist Mono (`--font-geist-mono`)
- **Titres** : `font-semibold tracking-tight`
- **Labels** : `text-sm font-medium`
- **Descriptions** : `text-xs text-muted-foreground`
- **Badges** : `text-[10px]` ou `text-[11px]`

11.3 Animations (Framer Motion)

- **Transitions de page** : `AnimatePresence mode="wait"`, fade + `translateY (8px)`
- **Pipeline stepper** : Progression de la ligne ambre, rotation du Loader2
- **Cartes** : `initial={{ opacity: 0, y: 10 }}` → `animate={{ opacity: 1, y: 0 }}`
- **Score circulaire** : `strokeDasharray` animé de 0 à la valeur cible
- **Checklist** : Stagger avec `delay: index * 0.03`

11.4 Responsive

- **Mobile-first** : Tous les composants s'adaptent.
- **Breakpoints** : `sm`: (640px), `md`: (768px), `lg`: (1024px)
- **Pipeline** : Horizontal sur desktop (`hidden md:block`), vertical sur mobile (`md:hidden`)
- **Grilles** : `grid-cols-1` → `sm:grid-cols-2` → `lg:grid-cols-3`
- **Header** : Navigation complète sur desktop, Sheet sur mobile

11.5 Règles de Layout

- **Footer sticky** : `min-h-screen flex flex-col` sur le wrapper + `mt-auto` sur le footer
 - **Max width** : `max-w-7xl mx-auto` pour le contenu principal
 - **Padding** : `px-4 py-6 sm:px-6 lg:px-8`
 - **Scroll custom** : Scrollbar stylisée pour les listes longues
-

12. Flux de Données End-to-End

12.1 Création de Session + Pipeline

```
[Utilisateur remplit le formulaire] | ▼ [Validation Zod côté client] | ▼ [POST /api/sessions] → [Prisma : Project.create + Session.create + 7× Step.create] | | | ▼ | [Response : session avec steps] | | → [pollStepStatus(sessionId)] ← Boucle asynchrone (toutes les 2s) | | | ▼ | [GET /api/sessions/{id}/steps/{n}] × 7 | | | ▼ | [Mise à jour du store : steps[], stepOutputs[], status] | ▼ [POST /api/sessions/{id}/execute] ← Non-bloquant | ▼ [Pour chaque étape 1→7 :] | | [UPDATE Step → status: 'running'] | | [UPDATE Session → currentStep: N] | | [ZAI.create() → zai.chat.completions.create()] | | messages: [system_prompt + accumulated_context] | | [UPDATE Step → status: 'completed', output, duration] | | [Si étape 6 : computeAndSaveValidation()] | | [Prisma : Validation.create] | ▼ [UPDATE Session → status: 'completed'] | ▼ [Response → parse score depuis output étape 6] | ▼ [Store : setValidationScore, setValidationDetails, setIsGenerating(false)]
```

12.2 Consultation d'une Session Passée

```
[Clic "Voir" sur une session] | ▼ [GET /api/sessions/{id}] → [Prisma : findUnique with steps + validations] | ▼ [Mapping des données dans le store :] | | setSteps(loadedSteps) | | setStepOutput(i, output) × N | | setValidationScore(score) | | setValidationDetails({ completeness, coherence, quality, checks }) | | setActiveTab("session") | ▼ [Affichage du pipeline + output + validation]
```

13. Règles d'Implémentation Inviolables

13.1 Règles Architecturales

1. **Un seul route visible** : Toute l'UI est dans `src/app/page.tsx` (route `/`). Jamais de routes additionnelles.
2. **API Routes uniquement** : Utiliser `app/api/` pour toute logique serveur. Pas de Server Actions.
3. **z-ai-web-dev-sdk côté serveur UNIQUEMENT** : Jamais d'import SDK côté client.
4. **Port 3000 unique** : Le dev server Next.js tourne exclusivement sur le port 3000.
5. ****Pas de `bun run build`**** : Utiliser uniquement `bun run dev` et `bun run lint`.

13.2 Règles de Code

6. **TypeScript strict** : Typage explicite pour toutes les interfaces, paramètres et retours.
7. **Composants shadcn/ui** : Toujours privilégier les composants existants (`src/components/ui/`). Ne jamais reconstruire.
8. **Imports** : Utiliser `@/` pour les chemins relatifs au projet.
9. **Pas de test code** : Ne jamais écrire de fichiers de test.
10. **Store comme source de vérité UI** : Toute la state UI passe par le Zustand store.

13.3 Règles UX/UI

11. **Pas de bleu/indigo** : Palette ambre/gold uniquement pour les éléments primaires.
12. **Footer sticky** : Toujours visible en bas, poussé naturellement par le contenu.
13. **Mobile-first responsive** : Touch targets ≥ 44 px, grilles adaptatives.
14. **Semantic HTML** : Utiliser `main`, `header`, `nav`, `footer`, `section`.
15. **Dark mode par défaut** : `defaultTheme="dark"` dans le ThemeProvider.
16. **Accessibilité** : `sr-only` pour les labels d'icônes, ARIA labels sur les boutons.

13.4 Règles Métier PEK

17. **Blocs A et H obligatoires** : Ils ne peuvent jamais être désélectionnés.
18. **Minimum 2 blocs** : Une session ne peut pas être lancée avec moins de 2 blocs.
19. **Chain-of-Thought vrai** : Chaque appel LLM reçoit les sorties accumulées des étapes précédentes. Jamais d'appels parallèles dans le pipeline.

20. **Scoring /25** : La validation est toujours sur 25 points (10 + 8 + 7).
21. **Langue française** : L'interface et les contenus par défaut sont en français.
22. **Pas de greenwashing** : Les prompts système interdisent les fausses promesses.
23. **4+ domaines d'exemples** : Chaque kit doit inclure des exemples variés dans au moins 4 domaines.

14. Glossaire Technique

Terme	Définition
PEK	Prompt-Engineering-Kit — Le produit lui-même
CoT	Chain-of-Thought — Méthodologie de raisonnement séquentiel en 7 étapes
Chaining	Accumulation de contexte entre les étapes (output N → input N+1)
Super-Agent	LLM configuré en tant qu'architecte autonome orchestrant le pipeline
Bloc	Module A-J composant un kit PEK
Session	Une exécution complète du pipeline pour un projet donné
Step	Une des 7 étapes du pipeline CoT
Validation	Évaluation qualité du kit généré (score /25)
Template	Modèle préconfiguré de sélection de blocs + system prompt
Polling	Mécanisme de vérification périodique du statut des étapes (toutes les 2s)
z-ai-web-dev-sdk	SDK pour les appels LLM (backend uniquement)
cuid	Identifiant unique collision-resistant (utilisé par Prisma)

15. Matrice de Diagnostic

Utilisez cette matrice pour diagnostiquer et résoudre les problèmes.

15.1 Problèmes Frontend

Symptôme	Cause Probable	Solution
Page blanche	Erreur d'import/hydratation	Vérifier <code>dev.log</code> , corriger les imports manquants
Onglets ne changent pas	<code>setActiveTab</code> non appelé	Vérifier le store Zustand et les Event Handlers
Pipeline ne se lance pas	Validation Zod échouée	Vérifier les min lengths (title:3, description:10, sourceContent:20)
Steps restent "pending"	Polling non démarré	Vérifier <code>pollStepStatus()</code> dans <code>pek-session-form.tsx</code>
Score non affiché	Regex de parsing échouée	Vérifier le format <code>{N}/25</code> dans l'output de l'étape 6
Footer flottant	Missing <code>min-h-screen flex flex-col</code>	Ajouter sur le wrapper racine dans <code>page.tsx</code>
Animations saccadées	Framer Motion key manquant	Vérifier <code>key={activeTab}</code> sur <code>AnimatePresence</code>

15.2 Problèmes Backend

Symptôme	Cause Probable	Solution
POST /api/sessions 500	Schema Prisma non synchronisé	Exécuter <code>bun run db:push</code>
LLM timeout	Réponse trop longue	Le SDK gère le timeout nativement
Score validation = 0	Aucun bloc détecté	Vérifier que <code>selectedBlocks</code> est bien en format "A,C,E"
Steps non créés	Erreur dans la création Prisma	Vérifier le <code>create</code> nested dans <code>sessions/route.ts</code>
Session stuck "in_progress"	Pipeline crashé silencieusement	Vérifier les logs serveur, le status doit passer à "error"

15.3 Problèmes de Données

Symptôme	Cause Probable	Solution
Anciennes sessions avec score null	Validation non encore implémentée	Le fallback dans le composant gère les null
Blocs obligatoires désélectionnés	<code>toggleBlock</code> bypassé	Vérifier la logique <code>if (config?.required) return state</code>
Store non réinitialisé	<code>resetPipeline()</code> non appelé	Toujours appeler avant une nouvelle session

Annexe A : Formule de Mapping Store ↔ DB

DB Status → Store Status ————— "in_progress" → "running" "completed" → "completed" "error" → "error" (pas de "pending" en DB pour session) → "pending" (défaut création)

Store Step Status → DB Step Status ————— "pending" → "pending" "active" → "running" "done" → "completed" "error" → "error"

Annexe B : Structure du Markdown Export

```
# Prompt-Engineering-Kit v4.0 – {titre} > Généré le {date complète} > Projet : {nom du projet} > Complexité : {complexité} | Langue : {langue} > Blocs sélectionnés : {blocs} --- ## Table des matières - [Étape 1 : COMPRÉHENSION](#...) - [Étape 2 : DÉCOMPOSITION](#...) - ... --- ## Contexte de la session {context} --- ## Étape 1 – COMPRÉHENSION > Statut : completed > Durée : 12s {output étape 1} --- ## Étape 7 – AMÉLIORATION {output étape 7} --- ## Rapport de validation ### Score global | Critère | Score | Maximum |
|-----|-----|-----| | Complétude | 8 | 10 | | Cohérence | 7 | 8 | | Qualité | 6 | 7 | | **Total** | **21** | **25** | **Note : B (Bon)** – ☐ Validé ### Vérifications détaillées - ☐ Complétude des blocs (4/5) - ☐ Exemples dans 4+ domaines - ... --- *Généré par Prompt-Engineering-Kit v4.0 – Méthodologie Chain-of-Thought en 7 étapes*
```

Fin du Méta-Prompt-Maître Ce document constitue la spécification vivante de PEK v4.0. Il doit être mis à jour pour toute modification architecturale, fonctionnelle ou technique de l'application.